

Word Embeddings

Overview

- outline the concept of representation learning and its importance in deep learning
- implement and apply word embeddings for natural language processing tasks

Async

Text

Representations
and Distance
Functions

- bag of words
 - represents text as a vector of counts of vocabulary words or binary values
- effective text representation models
 - leads to good task performance
 - results in semantically similar texts having similar representations

- distance functions
 - Euclidian Distance

$$\text{for } a, b \in \mathbb{R}^d \quad d(a, b) = \sqrt{\sum_{i=1}^d (a_i - b_i)^2}$$

- Cosine Similarity

$$\text{for } a, b \in \mathbb{R}^d \quad \text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|} = \cos \alpha$$

- small α : high similarity
- -1: opposing direction
- +1: same direction
- 0: orthogonal vectors

Word Embeddings/ Word2vec	- map each word to a vector in $\mathbb{R}^d$ such that similar words also have similar word vectors - this is used heavily in polymath applications such as OraCode
Skip-Gram Model	- given a word, predict its neighboring words within a window - example: is the word ‘brown’ likely to fall in the context window with ‘fox’ - objective to maximize: likelihood is product of $P(\text{context} \text{word})$ looping over all words in training data and their context words within a window of m $\mathcal{L}(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m; \\ j \neq 0}} P(w_{t+j} w_t; \theta)$ - we will maximize the log likelihood $J(\theta) = \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m; \\ j \neq 0}} \log P(w_{t+j} w_t; \theta)$ $= \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m; \\ j \neq 0}} \log \left(\frac{\exp(u_{w_t}^T v_{w_{t+j}})}{\sum_{w \in V} \exp(u_{w_t}^T v_{w_{t+j}})} \right)$ $= \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m; \\ j \neq 0}} u_{w_t}^T v_{w_{t+j}} - \log \sum_{w \in V} \exp(u_{w_t}^T v_{w_{t+j}})$ - gradient descent would be inefficient here, so: - set up task binary classification (positive and negative) - is ‘brown’ likely to appear with ‘fox’? - is ‘computer’ likely to appear with ‘fox’? - construct example pairs

	▪ consider a large text source – like Wikipedia - and set up positive examples by pairing target words ▪ create negative examples by randomly pairing other words
Continuous Bag of Words (CBOW) Model	• given the context, predict the word in the middle • model $P(\text{fox} \text{quick brown jumps over})$ • implementation similar to skip-gram • instead of pairs token, we have a token and a context consisting of multiple tokens • we handle this by representing the context with the sum of representations of all tokens in the context
Properties of Word Embeddings	• size of window affects the type of similarity • shorter window produces syntactically similar words ○ ex: Hogwarts -> Sunnysdale (fictional school) • longer window produces topically related words ○ ex: Hogwarts -> Dumbledore